

How far away?

Lesson 3 · Ultrasonic distance sensor + 5-LED bar

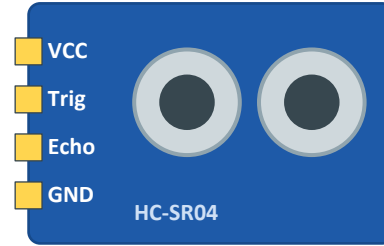
JA05 STEAM and Service Learning · 9 Oct 2026



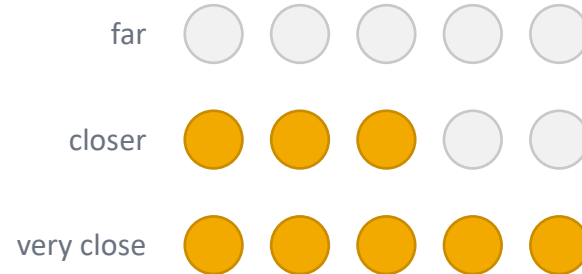
Today's build: a distance meter

Move your hand toward the sensor. The closer it gets, the more LEDs light up.

- Like the parking sensor on a car
- Like a bat finding its way in the dark
- Built in 6 small steps, each one tested



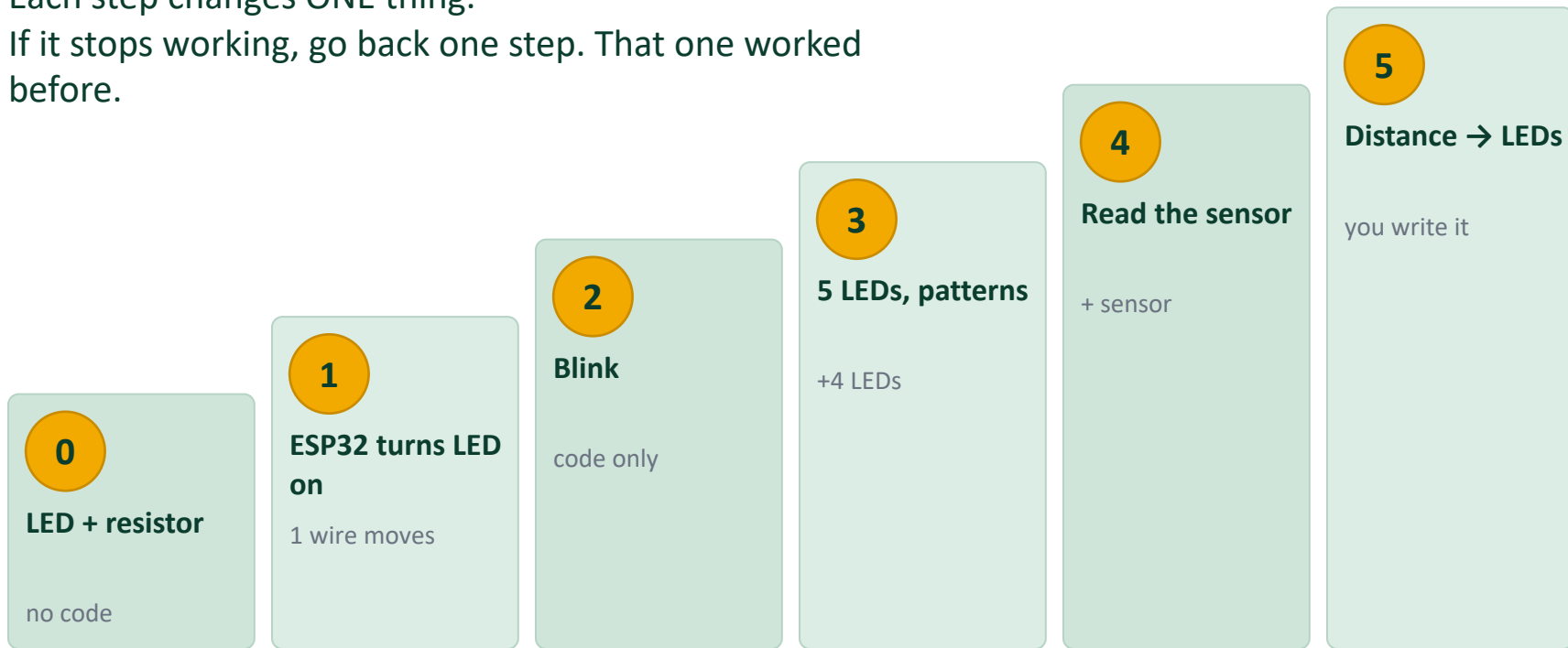
hand



6 small experiments, one at a time

Each step changes ONE thing.

If it stops working, go back one step. That one worked before.



Get today's files

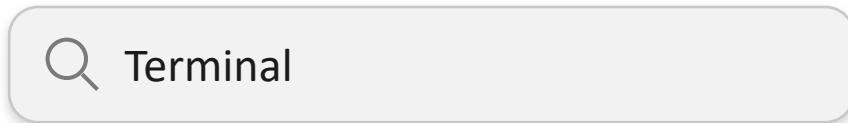
Lost your JA05 folder, or need the newest files? Start here.

Open Terminal

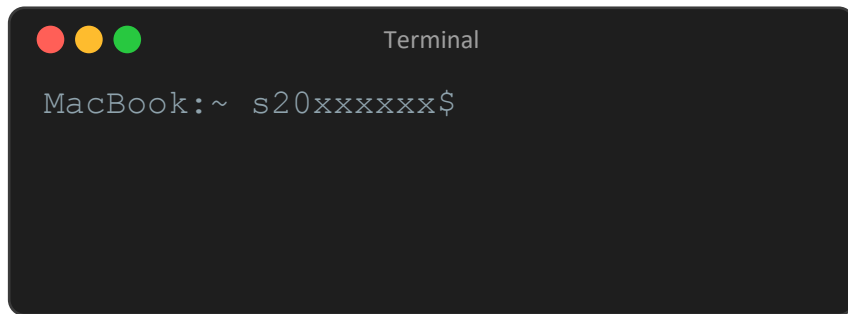
- 1 Press  **Command + Space**
- 2 Type **Terminal**
- 3 Press **return**

CHECKPOINT A window with a \$ at the end of the line.

~ (tilde) means your home folder. That is where JA05 lives.



Spotlight search



Do you already have JA05?

```
Terminal
MacBook:~ s20xxxxxx$ cd
MacBook:~ s20xxxxxx$ ls
Desktop    Documents  Downloads
JA05       Movies     Music
```

cd go to your home folder

ls list what is inside

JA05 is in the list

Update it with git pull. Next slide.

No JA05

Download it again with git clone.
Slide after next.

Have JA05? Get the latest

```
Terminal
MacBook:~ s20xxxxxx$ cd ~/JA05
MacBook:JA05 s20xxxxxx$ git pull
Updating 0d04113..8c1f2a4
Fast-forward
 20261009_L3/step1_led_on.py | 17 +
 PPTs/20261009_L3.pptx      | Bin
```

`cd ~/JA05` go into the JA05 folder

`git pull` download the newest files from GitHub

CHECKPOINT You see Fast-forward, or Already up to date.

Already up to date.

Good news: you already have everything. Nothing to do.

STUCK? Error? See the last slide of this part.

No JA05? Download it again

Terminal

```
MacBook:~ s20xxxxxx$ cd
MacBook:~ s20xxxxxx$ git clone https://github.com/jackyhui/JA05.git
Cloning into 'JA05'...
done.
MacBook:~ s20xxxxxx$ cd JA05
```

git clone makes a new JA05 folder and copies every file from GitHub into it.

Only once. After this, use git pull every lesson.

CHECKPOINT Type ls again. JA05 is now in the list.

Find today's files

```
Terminal
MacBook:~ s20xxxxxx$ open ~/JA05
```

Opens the JA05 folder in Finder.

Open today's slides straight away:

```
open ~/JA05/PPTs/20261009_L3.pptx
```

In VS Code: File > Open Folder > JA05.

JA05

20260918_L1

Lesson 1 code

20261009_L3

today's code

step1_led_on.py ...

PPTs

20260918_L1.pptx

Lesson 1 slides

20261009_L3.pptx

today's slides

Terminal error? Read the message

You see	What it means	Do this
destination path 'JA05' already exists	You already have JA05	<code>cd ~/JA05</code> then <code>git pull</code>
not a git repository	You are in the wrong folder	<code>cd ~/JA05</code> first
No such file or directory	Typo in the name	JA05 = J A zero five
Your local changes ... would be overwritten	You edited a file that changed online	<code>cd ~</code> then <code>mv JA05 JA05_old</code> then <code>git clone</code> again
Could not resolve host: github.com	No internet	Join Wi-Fi <code>css_STEAM</code> , try again

`mv JA05 JA05_old` keeps your old work safe in `JA05_old`. Copy anything you need from there.

Materials for today



ESP32 + extension board

from Lesson 1



Breadboard + jumper wires

red, black, and colours



5 LEDs

any colour



5 resistors, 220 Ω

red-red-brown bands



HC-SR04 ultrasonic sensor

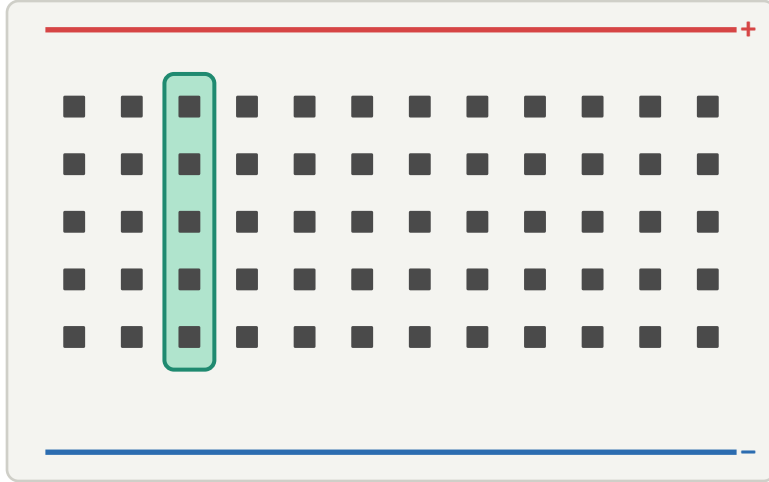
the one with two 'eyes'



USB cable

computer to ESP32

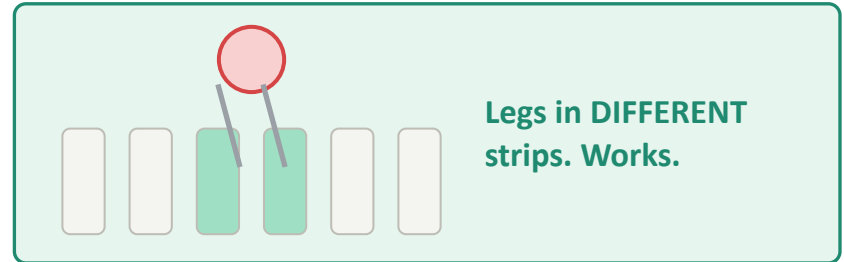
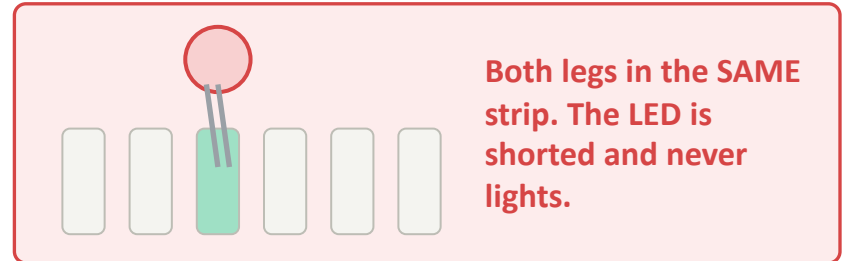
Breadboard: which holes connect?



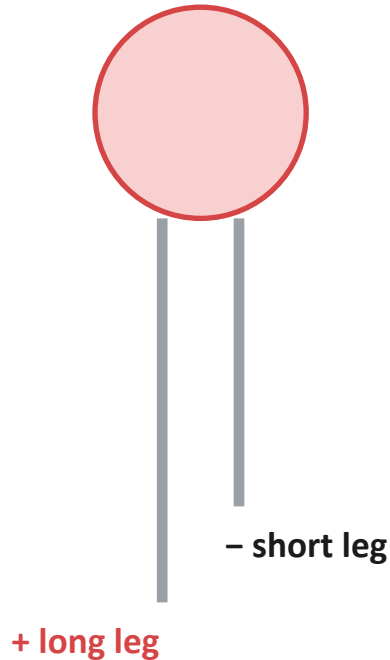
These 5 holes are joined inside

Each short line of 5 holes is one connection. The long + and – lines along the edge are rails, often used for power and GND.

Most common mistake



LED + resistor: the rules



Long leg (+) toward power or the GPIO pin.

Backwards = no light. Flip it, nothing breaks.

Short leg (-) goes to GND.

GND is the 'return path' that closes the loop.

Always add a 220 Ω resistor.

No resistor = too much current. The LED or the pin can burn out.

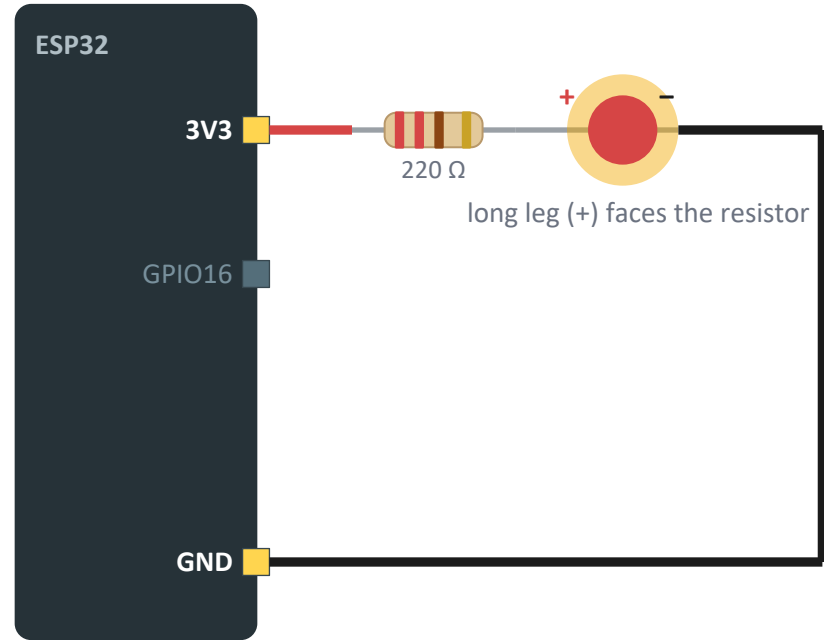
Step 0: light an LED with no code



1. Plug the ESP32 into the breadboard.
2. Red wire: 3V3 pin → resistor.
3. Resistor → LED long leg (+).
4. Black wire: LED short leg (-) → GND.
5. Plug in the USB cable.

CHECKPOINT LED is ON. No code needed.

STUCK? Flip the LED. Check the breadboard slide.



Step 1: let the ESP32 turn it on

1

Move ONE wire: from 3V3 to GPIO16.
Everything else stays.

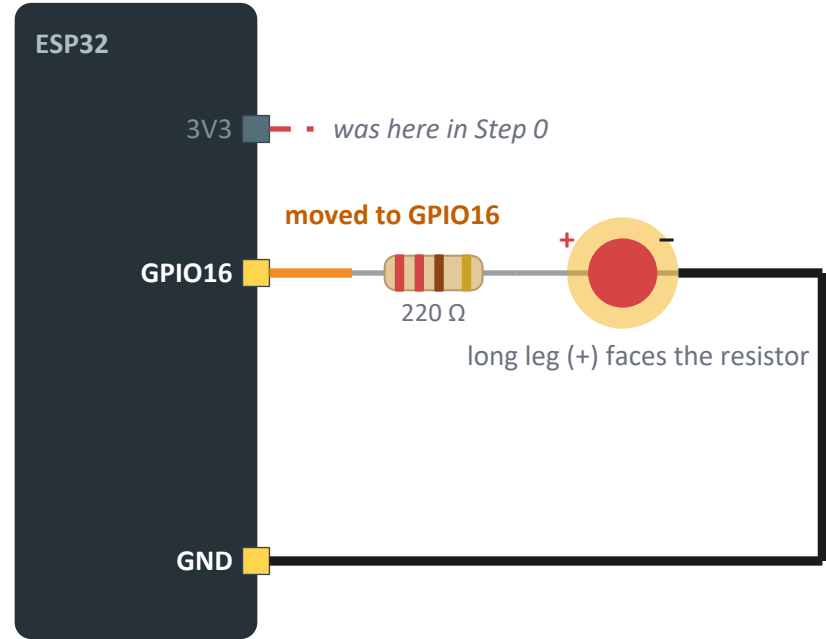
```
from machine import Pin

led = Pin(16, Pin.OUT)
led.value(1)      # 1 = ON
```

File: step1_led_on.py · Try value(0)

CHECKPOINT LED turns ON after you run the code.

STUCK? Wire back to 3V3. Does Step 0 still work?



Step 2: blink

2

No wiring change. Only the code changes.

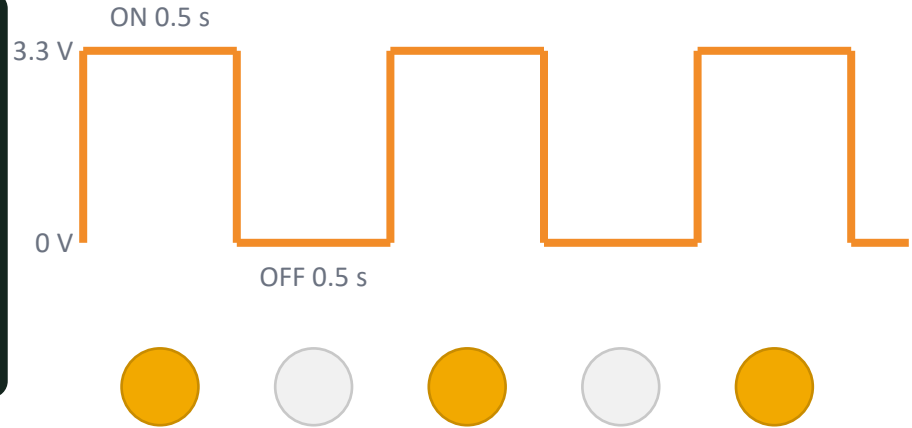
```
led = Pin(16, Pin.OUT)

while True:
    led.value(1)
    time.sleep(0.5)
    led.value(0)
    time.sleep(0.5)
```

File: step2_led_blink.py · Ctrl+C to stop

CHECKPOINT LED blinks twice a second.

What the pin does



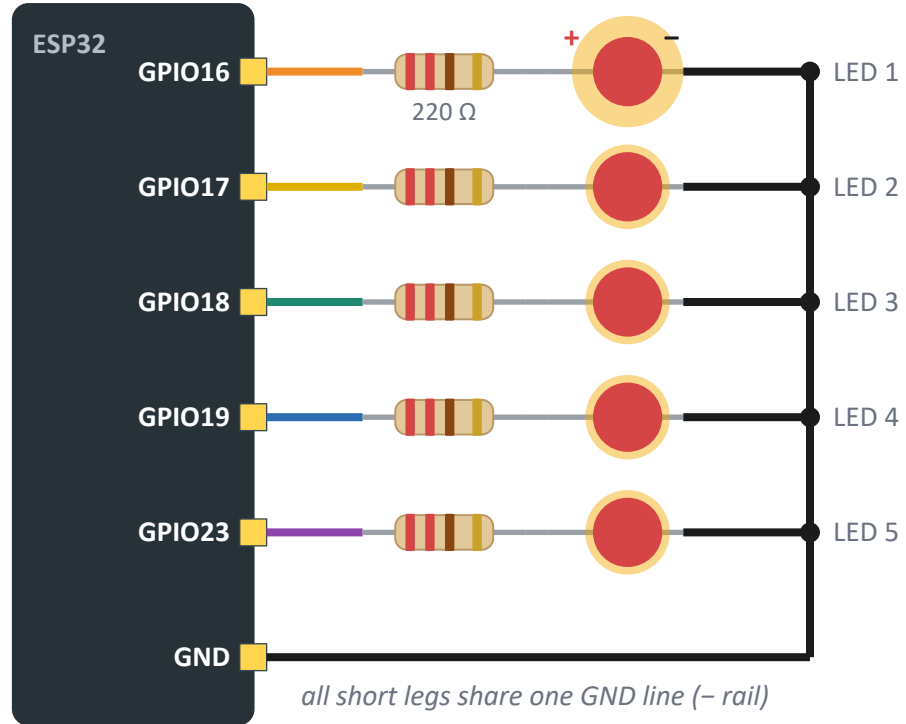
Try 0.1 instead of 0.5. What changes?

Step 3: copy the circuit 4 more times

3

- LED 1 is your Step 2 circuit. Keep it.
- Add LEDs 2 to 5, one at a time.
- Use the – rail for GND: one black wire from GND to the rail, then each short leg to the rail.

CHECKPOINT Run `step3a_check_leds.py` next.



Step 3a: test each LED

3a

```
PINS = [16, 17, 18, 19, 23]
leds = [Pin(p, Pin.OUT) for p in PINS]

for led in leds:
    led.value(1)
    time.sleep(1)
    led.value(0)
```

File: step3a_check_leds.py (prints which LED is on)

CHECKPOINT LED 1 to 5 light up one after another.

One LED stays dark? Fix only that one.

- 1 Long leg on the resistor side?
- 2 Short leg reaches the – rail?
- 3 Wire in the right GPIO pin?
- 4 Legs in different strips?
- 5 Swap with an LED that works.

Step 3b: make a pattern

```
show([1, 0, 1, 0, 1])  
time.sleep(0.5)  
show([0, 1, 0, 1, 0])
```

1 = ON, 0 = OFF. One number for each LED.

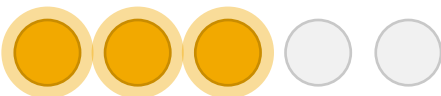
File: step3b_led_patterns.py has chase, bar and blink.
Add your own pattern at the bottom of the loop.

CHECKPOINT Your own pattern runs.

3b

[1, 0, 1, 0, 1] 

[0, 1, 0, 1, 0] 

bar(3) 

bar(5) 

Part 2: the ultrasonic sensor

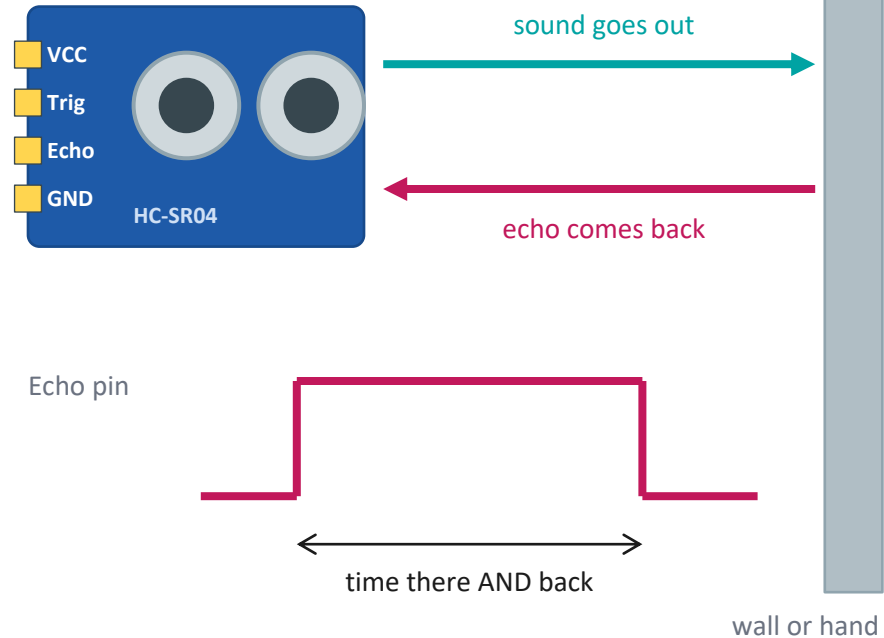
Your LEDs work. Keep them plugged in.

How it works: shout and listen

Trig: the ESP32 tells the sensor to send a short 'click' of sound. Too high for us to hear (40 kHz).

Echo: the pin stays ON until the sound bounces back.

Longer Echo = object further away.



Bats and dolphins find things the same way.

You have seen this before



Car parking sensor

Beeps faster as the car gets close to a wall.

Photo: Shixart1985, CC BY 2.0



Car park space finder

One sensor over each space. Red light = taken, green = free.

Photo: Z22, CC BY-SA 4.0



Ultrasound scan

Doctors see inside the body using echoes.

Photo: Goleisureintl, CC BY 4.0



Bats

Find insects in the dark by listening for echoes.

Photo: Chanut F, PLoS Biol, CC BY 2.5

A car does what we are building

Car

sensor



brain

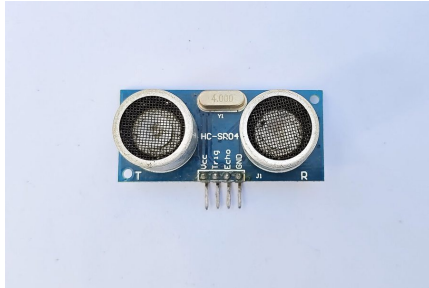
Car computer

output



Photos: Shixart1985, CC BY 2.0 · Nozilla, CC BY-SA 3.0

Ours



ESP32

5 LEDs



Photo: Suyash Dwivedi, CC BY-SA 4.0

From time to distance

$$\text{distance} = \text{time} \times \text{speed of sound} \div 2$$

343 m/s

speed of sound in air
= 0.0343 cm per μs

$\div 2$

the sound goes there
AND back

1000 μs

$1000 \times 0.0343 \div 2$
= 17.2 cm

In code: `return duration * 0.0343 / 2`

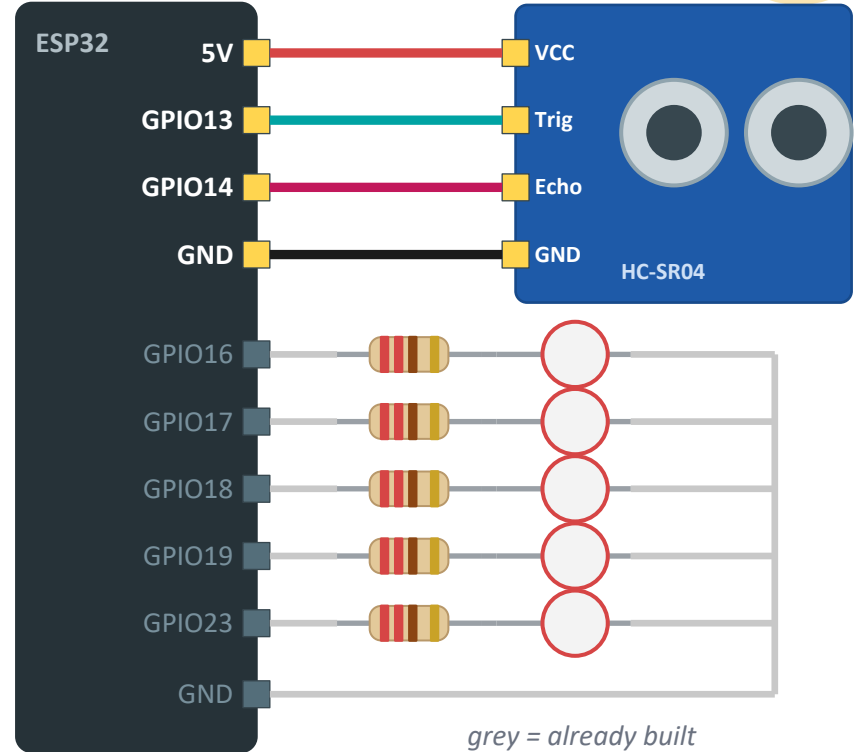
Step 4: add the sensor

4

Sensor pin	ESP32 pin	Wire
VCC	5V	red
Trig	GPIO13	any colour
Echo	GPIO14	any colour
GND	GND	black

Read the pin names printed on the sensor. VCC needs 5V, not 3V3.

STUCK? Unplug the sensor. Does step3b still run?



Step 4: read the distance

4

```
trig.value(1)
time.sleep_us(10)      # 10 µs 'click'
trig.value(0)

duration = time_pulse_us(echo, 1, 30000)
if duration < 0:
    return None        # no echo
return duration * 0.0343 / 2
```

File: step4_ultrasonic_print.py

CHECKPOINT The number changes when you move your hand.

Shell

```
Distance: 24.6 cm
Distance: 18.1 cm
Distance: 11.3 cm
Distance: 6.0 cm
```

time_pulse_us waits for Echo to go ON, then times how long it stays ON. 30000 µs is the time limit (about 5 m).

Part 3: distance → LEDs

No new wires. Only code from here.

Step 5: you write the rule

5

25 cm or more



20 – 25 cm



15 – 20 cm



10 – 15 cm



5 – 10 cm



under 5 cm



```
def leds_for(distance):  
    if distance < 5:  
        return 5  
    elif distance < 10:  
        return ...
```

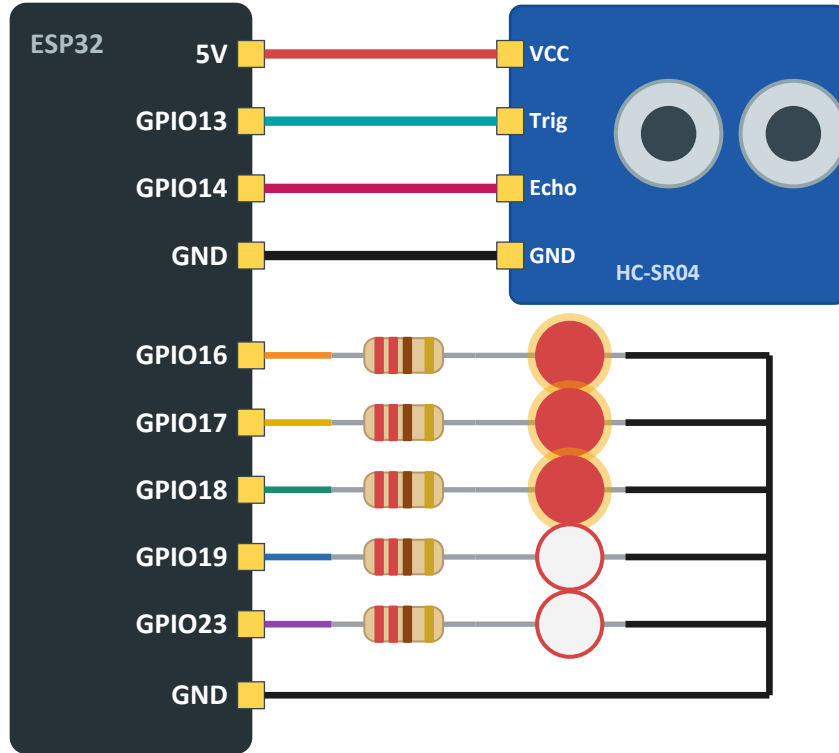
Finish `leds_for()` so it matches every row of the table. It must return a number from 0 to 5.

File: `step5_distance_leds.py`

CHECKPOINT Shell shows the same LED count as the table.

STUCK? Ask your teacher for the finished file.

The whole circuit



Part	ESP32 pin
LED 1 – 5	16, 17, 18, 19, 23
Sensor VCC	5V
Sensor Trig	GPIO13
Sensor Echo	GPIO14
All GND	GND / – rail

Pin positions in the drawing are not where they sit on your board. Read the labels on the extension board.

Stuck? Go back one step

What you see	Go back to	Check
No LED lights at all	Step 0	LED direction, GND wire, breadboard strips
Step 0 works, Step 1 doesn't	Step 1	Wire in GPIO16, not a pin next to it
One of 5 LEDs dark	Step 3a	That LED's legs, resistor, GPIO pin
'No echo' every time	Step 4	Trig ↔ Echo swapped? VCC on 5V?
Distance jumps around	Step 4	Point at a flat, hard surface
Board resets / won't connect	—	Unplug USB, check for a short to GND

Challenge: a parking sensor

6

Paste your `leds_for()` from Step 5 into `step6_challenge_parking_sensor.py`, then run it. All LEDs flash when something is closer than 5 cm.

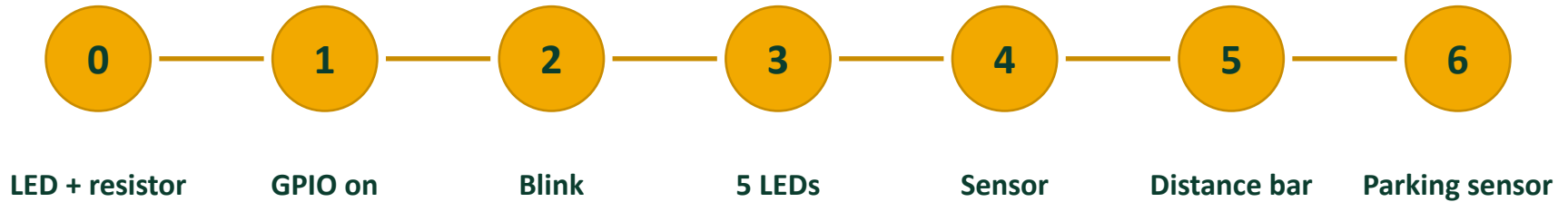
- Flash faster the closer you get
- Green, yellow, red LEDs
- Add the buzzer from the kit

Service learning

Who in our community could use a distance warning?

- Someone with low vision walking indoors
- A wheelchair user reversing
- A shop door that counts visitors

What did you reach today?



- A GPIO pin is a switch your code controls.
- The sensor measures time; maths turns it into distance.
- Build small, test, then add one thing.